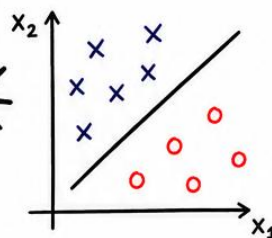
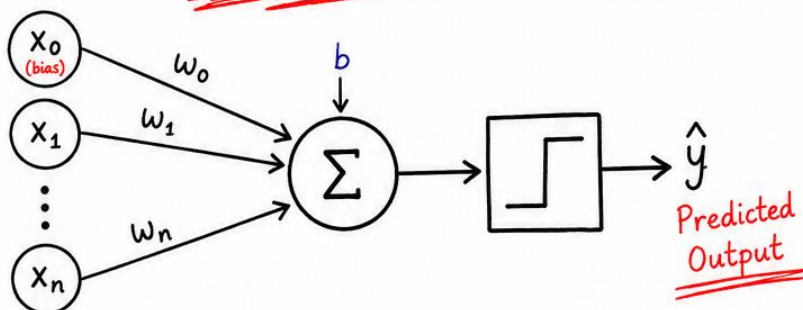


DEEP LEARNING



PERCEPTRON TRICK



TOPIC NUMBER : 01

📖 TOPIC : Deep Learning

👤 CREATED BY : Shahin

🏫 CSE , Daffodil International University

🐙 GITHUB : https://github.com/shahin5646/Deep_learning

The simplest ideas build the smartest models. ★



THINK

→ LEARN
→ UPDATE
→ PREDICT

Deep Learning

Date: / /

Learning about Penceptron:

Rough Ideas

- Penceptron → used in supervised machine learning
- Activation function → Output value, Range এর মধ্যে
 - ↳ Training process এর main objective, হলো weight & bias এর value বের করা,
- * Neuron vs Perceptron → Perceptron weakly inspired from neuron.

weight: Feature Importance

Perceptron is a binary classifier

$Ax + by + c$ → Equation of line

→ it divide data by creating two regions

or বা, যদি 3 টি input হয় like IQ, cgpa, mark

Then the Equation will be: $w_1x_1 + w_2x_2 + w_3x_3 + b > 0$

Equation of
a plane

$$Ax_1 + By + Cz + b \geq 0$$

$$Ax + By + Cz + b \geq 0$$

hyperplane,	2D	perceptron	→ line
	3D	perceptron	→ plane
hyperplane,	4D or more		→ hyperplane
Data			

Perceptron can be used Linear/sort of Linear Data

Perceptron Trick: How to train a perceptron

Usually perceptron trick এ Loop চালানো হয়। Usually, it's **thousand times**. একটি random point select করা হয়। তারপর loop একটি point select করবে and ask করবে যে এটা কি সঠিকভাবে classified? If not, তাহলে Line move করবে। আর যদি সঠিকভাবে classified হয়, যেমন আছে ঠিক সেভাবেই থাকবে।

Loop কতক্ষণ চলবে?

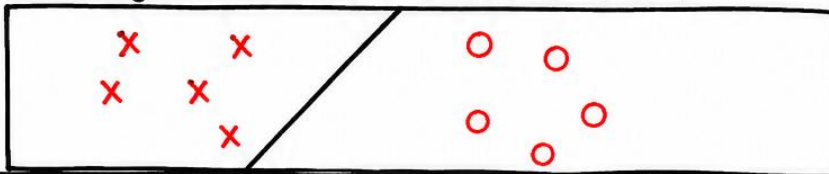
- Generally thousand (1000) times ও result চলে আসে বা 10000 ও। এটা চালানো **still convergence**।
- প্রত্যেকবার check করা কী পরিমাণ **misclassified**।
- এপর্যন্ত চালানো যতক্ষণ না misclassified point এর সংখ্যা **0** হয়। Then **stop the Loop**.

(Ful) The number of times Loop runs also called **Epoch**

কিভাবে line move করে বা Transformation হবে?

How to label regions?

- কিভাবে decide করা হবে কোন region **+ve positive** আর কোন region **Negative**?
- কোন region কোন **side** থাকবে লাইনের?



Update Needed

The handwriting is mostly readable, but a few words may require verification from the original note:

- ① "still convergence"
- ② "(Ful) The number of times loop runs also called Epoch"
- ③ "কিভাবে line move করে বা Transformation হবে?"

These have been transcribed as closely as possible without changing the original wording or meaning.

Line equation, $Ax + By + C = 0$

যদি দুটি point x_1, y_1 হয় তাহলে,

- # $Ax_1 + By_1 + C > 0$ হলে **Positive** region.
- # $Ax_1 + By_1 + C < 0$ হলে **Negative** region.
- # $Ax_1 + By_1 + C = 0$ হলে Line এর উপর,

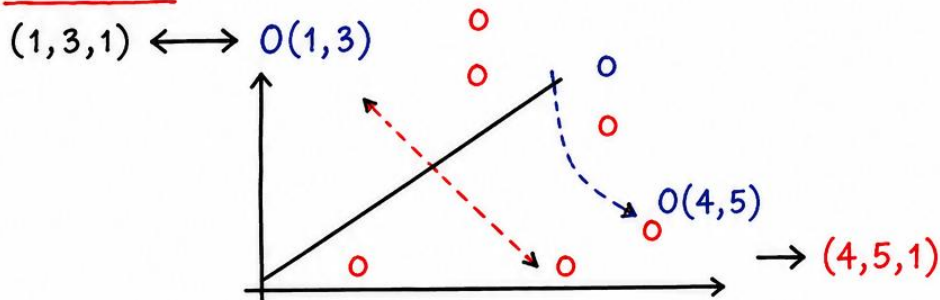
How to move / Transform Line

Movement mainly আসে A, B, C তে changes করার মাধ্যমে,

- # C তে changes এর মাধ্যমে Parallely উপরে নিচে যায় Line।
 C বাড়লে নিচের দিকে যায়, কমাতে উপরের দিকে যায়।
- # A তে changes এর মাধ্যমে line এর slope change হয় এবং rotation হয়।
- # B তে changes এর মাধ্যমে line এর slope change হয় এবং rotation হয়।

Practical : Use Desmos. → Line coef : 2, 3, 5

Transform



(+)

2	3	5
1	3	1

$$3x + 6y + 6 \geq 0$$

(-)

2	3	5
4	5	1

$$-2x - 2y + 4 \leq 0$$

Line **Negative** region এর দিকে move করলে (-)

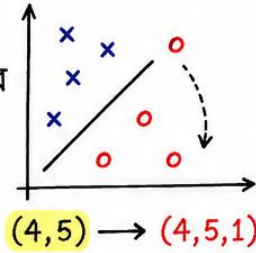
Positive region এর দিকে move করলে (+)

Summary :

যদি কোনো Negative point Positive region এ হয় তাহলে
ঐ point এর co-ordinate এর সাথে bias input 1 add করে

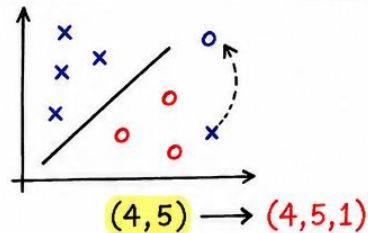
e.g : Original (4,5)

Transformed : (4,5,1) করে subtract (-) করে
Line move করা হয়।



যদি Positive point Negative Region এ হয়

তখন add (+) করে Transform করতে
লাইন move করতে হয়।



Generally Machine Learning এ আমরা weight update করার সময়
transformation করি। আমরা generally point এর co-ordinate এর
সাথে learning rate multiply করে add অথবা subtract করি।

New Coefficient = Old Coefficient - $\eta \times$ Co-ordinate (for negative update)

New Coefficient = Old Coefficient + $\eta \times$ Co-ordinate (for positive update)

η (eta) means Learning Rate.

Question : Why we add 1 ?

Answer : আমরা point এর সাথে 1 add করি Bias term represent করার জন্য।

যেমন,

Original Point = (4,5)

After Bias Addition = (4,5,1)

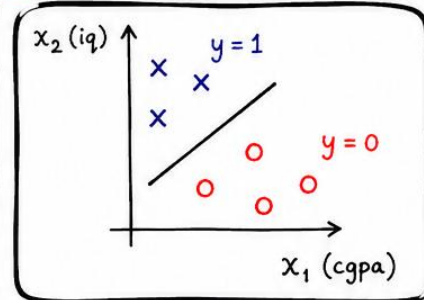
$$(4,5) \xrightarrow{\text{2D point}} \begin{bmatrix} 4 \\ 5 \end{bmatrix} \xrightarrow{\text{add 1}} \begin{bmatrix} 4 \\ 5 \\ 1 \end{bmatrix} \xrightarrow{\text{3D (with bias)}}$$

- # এখানে শেষের 1 হলো bias input, যার মাধ্যমে perceptron decision boundary (line) কে shift করতে পারে।
- # Bias না থাকলে line সবসময় origin (0,0) এর মধ্য দিয়ে যেতে বাধ্য হবে।
- # Bias থাকলে line origin ছাড়া অন্য যেকোনো position এ move বা shift করা সম্ভব হয়।

Algorithm

Dataset

x_0 (bias)	x_1 (cgpa)	x_2 (iq)	y (placed)
1	7.5	81	1
1	8.9	109	1
1	7	81	0



Line Equation

$$Ax + By + C = 0$$

Perceptron Equation :

$$w_0 + w_1 x_1 + w_2 x_2 \geq 0$$

যেহেতু x_0 এর value = 1, তাই equation লেখা যায়,

$$w_0 x_0 + w_1 x_1 + w_2 x_2 \geq 0$$

অথবা,

$$\sum_{i=0}^2 w_i x_i \geq 0$$

Prediction Rule

If

$$w_0 x_0 + w_1 x_1 + w_2 x_2 > 0$$

Then,

$$y = 1 \text{ (Placed)}$$

Otherwise,

$$y = 0 \text{ (Not Placed)}$$

Relation between Line Equation and Weights

$$w_0 = C$$

$$w_1 = A$$

$$w_2 = B$$

So,

$$Ax + By + C = 0$$

$\downarrow$
 w_1

$\downarrow$
 w_2

$\downarrow$
 w_0

How the model predict ?

If we have w_0, w_1, w_2 তাহলে prediction কিভাবে হবে ?

For first row, তাহলে,

$$x_0 = 1$$

$$x_1 = 7.5$$

$$x_2 = 81$$

$$w_0 x_0 + w_1 x_1 + w_2 x_2$$

=

$$w_0 \times 1 + w_1 \times 7.5 + w_2 \times 81$$

এখানে থেকে একটি result আসবে।

যদি,

$$\text{Result} > 0$$



তাহলে output = 1
অর্থাৎ Placement হবে।

আর যদি,

$$\text{Result} < 0$$



তাহলে output = 0
অর্থাৎ Placement হবে না।

Example (First Row)

Input:

$$x_0 = 1$$

$$x_1 = 7.5$$

$$x_2 = 81$$

Prediction Score :

$$w_0(1) + w_1(7.5) + w_2(81)$$

• If Score > 0 → Placed

• If Score < 0 → Not Placed

Note : এখানে $x_0 = 1$ হলো bias input, যা decision boundary (line) কে shift করতে সাহায্য করে।

$$w = [w_0, w_1, w_2]$$

(Weight Vector)

Algorithm

আমরা basically dot product perform করি,

$$\begin{bmatrix} w_0 & w_1 & w_2 \end{bmatrix} \times \begin{bmatrix} x_0 \\ x_1 \\ x_2 \end{bmatrix} = w_0 x_0 + w_1 x_1 + w_2 x_2$$

Algorithm

Epoch = 1000 , Learning Rate , $\eta = 0.01$

Loop

for i in range (epochs):

- যদি, $x_i \in \text{Negative Region}$ and $\sum_{i=0}^2 w_i x_i > 0$ তাহলে,

$$w_{\text{new}} = w_{\text{old}} - \eta \times x_i$$

- যদি, $x_i \in \text{Positive Region}$ and $\sum_{i=0}^2 w_i x_i < 0$ তাহলে,

$$w_{\text{new}} = w_{\text{old}} + \eta \times x_i$$

Meaning :

- যদি আমাদের point Negative Region এ belong করে কিন্তু model সেটিকে Positive Region হিসেবে predict করে , তাহলে বিয়োগ করে update করতে হবে।
অর্থাৎ, $w_{\text{new}} = w_{\text{old}} - \eta \times x_i$

- আর যদি point Positive Region এ belong করে কিন্তু model সেটিকে Negative Region হিসেবে predict করে , তাহলে যোগ করে update করতে হবে।
অর্থাৎ, $w_{\text{new}} = w_{\text{old}} + \eta \times x_i$

Perceptron Update Rule (Summary)

False Positive
(Actual Negative, Predicted Positive)

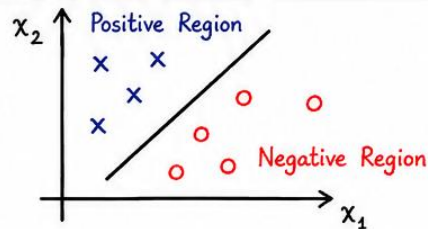
$$w_{\text{new}} = w_{\text{old}} - \eta x_i$$

False Negative
(Actual Positive, Predicted Negative)

$$w_{\text{new}} = w_{\text{old}} + \eta x_i$$

যেখানে,

- η (eta) = Learning Rate
- x_i = Input Feature Vector
- w = Weight Vector $[w_0, w_1, w_2]$



(i) যদি

$$x_i \in N \text{ and } \sum w_i x_i \leq 0$$

Update হবে না।

(ii) যদি

$$x_i \in P \text{ and } \sum w_i x_i > 0$$

No update needed.

Simplify The Algorithm

If, $x_i \in N$ and $\sum w_i x_i > 0$

$$\Rightarrow w_n = w_o - \eta x_i$$

If, $x_i \in P$ and $\sum w_i x_i < 0$

$$\Rightarrow w_n = w_o + \eta x_i$$

Notation:

N = Negative Region

P = Positive Region

x_i = Input Feature Vector

w_o = Old Weight Vector

w_n = New Weight Vector

η = Learning Rate

Updated Simplified

$$w_n = w_{old} + \eta (y_i - \hat{y}_i) x_i \rightarrow$$

Justification

Y_i	$\hat{Y}_i$	$Y_i - \hat{Y}_i$
1	1	0
0	0	0
1	0	1
0	1	-1

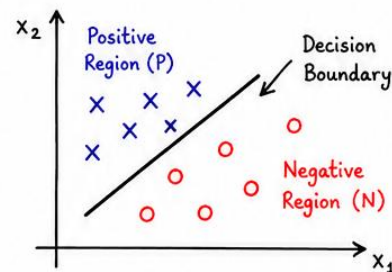
Context-Based Updated Version

Using the justification table, the two separate update rules can be combined into a single **perceptron learning rule**:

$$w_{new} = w_{old} + \eta (y_i - \hat{y}_i) x_i$$

Where:

- w_{new} = Updated weight vector
- w_{old} = Previous weight vector
- η = Learning rate
- x_i = Input feature vector
- y_i = Actual label
- $\hat{y}_i$ = Predicted label



Interpretation

★ If $y_i = \hat{y}_i$, then $(y_i - \hat{y}_i) = 0 \rightarrow$ No update needed

★ If $y_i = 1$ and $\hat{y}_i = 0$, then $(y_i - \hat{y}_i) = 1 \rightarrow$ Add ηx_i

★ If $y_i = 0$ and $\hat{y}_i = 1$, then $(y_i - \hat{y}_i) = -1 \rightarrow$ Subtract ηx_i

Key Point:

We update the weights only when the model predicts the **wrong region** for the given point.

Code Algo:

```
for i in range (epochs)
    select a random student (i)
```

$$W_n = W_o + \eta (y_i - \hat{y}_i) \times X_i$$

Symbol Clarifications (Update Needed)

- η (eta) = Learning rate.
- $\hat{y}_i$ (y -hat i) = Predicted output.
- y_i = Actual / True label.
- X_i = Selected student's feature vector.

Formula Interpretation

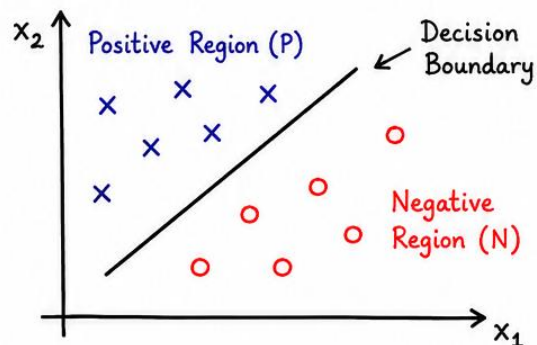
So, the formula can be interpreted as the standard **perceptron update rule**:

$$W_{\text{new}} = W_{\text{old}} + \eta (y_i - \hat{y}_i) X_i$$

Interpretation

- ★ If $y_i = \hat{y}_i$, then $(y_i - \hat{y}_i) = 0$
→ No update needed.
- ★ If $y_i = 1$ and $\hat{y}_i = 0$,
then $(y_i - \hat{y}_i) = 1$
→ Add ηX_i
- ★ If $y_i = 0$ and $\hat{y}_i = 1$,
then $(y_i - \hat{y}_i) = -1$
→ Subtract ηX_i

Visual Understanding



Note: Other than these symbol clarifications,
the handwritten content has been transcribed exactly as written.